



ІТМО

Разработка инструмента планирования миграции кодовой базы проекта на алгоритмы постквантовой криптографии

Студент:

Пронина Анастасия Ильинична, Р3433

Руководитель:

Рыбаков Степан Дмитриевич, аспирант и
преподаватель практики факультета ПИиКТ

Актуальность: квантовая угроза и сроки

Алгоритм Шора взламывает за полиномиальное время:

- RSA, DSA, DH;
- ECDH, ECDSA;
- ГОСТ Р 34.10-2012.

Прогнозы появления CRQC:

- Global Risk Institute (2025/2026):
28-49% вероятность в ближайшие 10 лет;
- Project Eleven: “Q-Day” $\approx$ **2033**;
- Оценка ресурсов на взлом RSA-2048:
падение с 20 млн. (2019) до 100 тыс. (2026)
физических кубитов.

2024

NIST FIPS 203/204/205

2030

Устаревание RSA-2048,
ECDSA P-256

2033

Q-Day

2035

Полный запрет квантово-
уязвимых алгоритмов

Актуальность: инвентаризация и срочность

Масштаб проблем в исходном коде:

- С и C++ — **2 и 3** места индекса TIOBE (апрель 2026);
- основные крипто-библиотеки реализованы на C/C++;
- тысячи вызовов на десятки файлов в типичном проекте.

Готовность отрасли (IBM IBV & CSA, 2025):

- Лишь **30%** организаций завершили инвентаризацию криптографических активов;
- Только **24%** из них начали применять ее результаты в контексте перехода на постквантовую криптографию.

NIST IR 8547 – миграция требует времени:

- **2030:** устаревание 112-битных алгоритмов
- **2035:** полный запрет в стандартах FIPS
- НПК «Криптонит»: разработка и внедрение крипто-механизмов занимает десятилетия, а не годы.

HNDL: "Harvest Now, Decrypt Later"

Противник архивирует зашифрованный трафик уже сейчас, рассчитывая расшифровать его после появления криптографически значимого квантового компьютера.

Цель и задачи

Цель: автоматизация процесса оценки и планирования миграции к постквантовым криптографическим алгоритмам посредством выявления кванто-уязвимых решений и формирования плана их модернизации на основе стандартов NIST с учетом развития отечественной нормативной базы ТК 26.

Задачи:

1. Провести анализ квантовых угроз, изучить профильные стандарты, выявить ограничения существующих аналогов.
2. Спроектировать модульную архитектуру инструмента и базу сигнатур.
3. Реализовать модули инвентаризации файлов проекта, извлечения вызовов криптографических функций и анализа сертификатов.
4. Реализовать модули оценки рисков и планирования миграции.
5. Провести функциональное тестирование и оценить точность обнаружения.

Анализ аналогов: полностью рабочих решений нет

Инструмент	Поддержка C/C++	ТК 26 / ГОСТ	План миграции	NIST IR 8547
IBM Quantum Safe Explorer	Только API Discovery	—	Частично	Частично
CodeQL	Да	—	—	—
CBOMkit	Частично	—	—	—
CryptoScan	Частично	—	—	—
SonarQube	Частично	—	—	—
cdxgen	—	—	—	—
QApp	—	Да	Услуги	—

Вывод: ни один из инструментов не реализует полноценной цепочки «обнаружение – оценка риска – формирование плана миграции». ГОСТ-сигнатуры не поддерживаются.

Архитектура: модульный конвейер

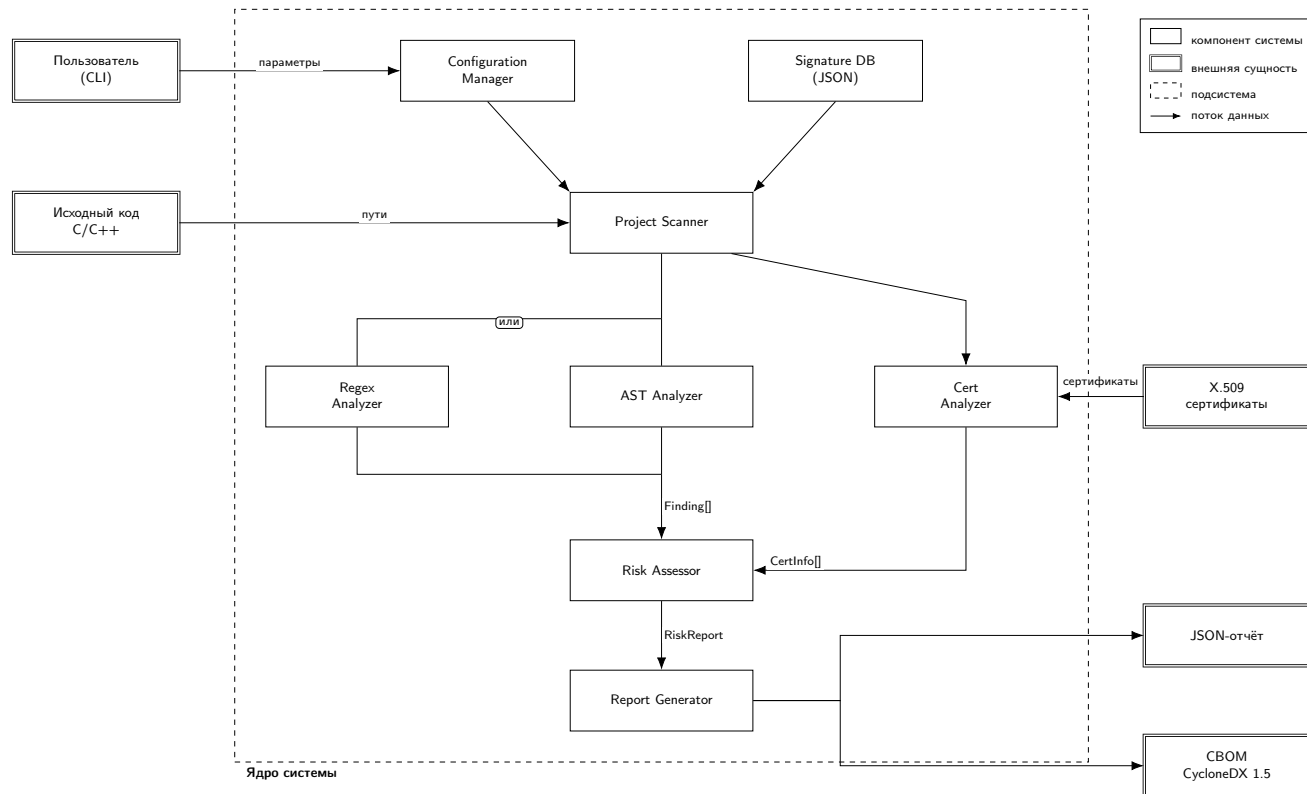


Рисунок 1 – Компонентная архитектура инструмента миграции на PQC

Архитектура: процесс выполнения

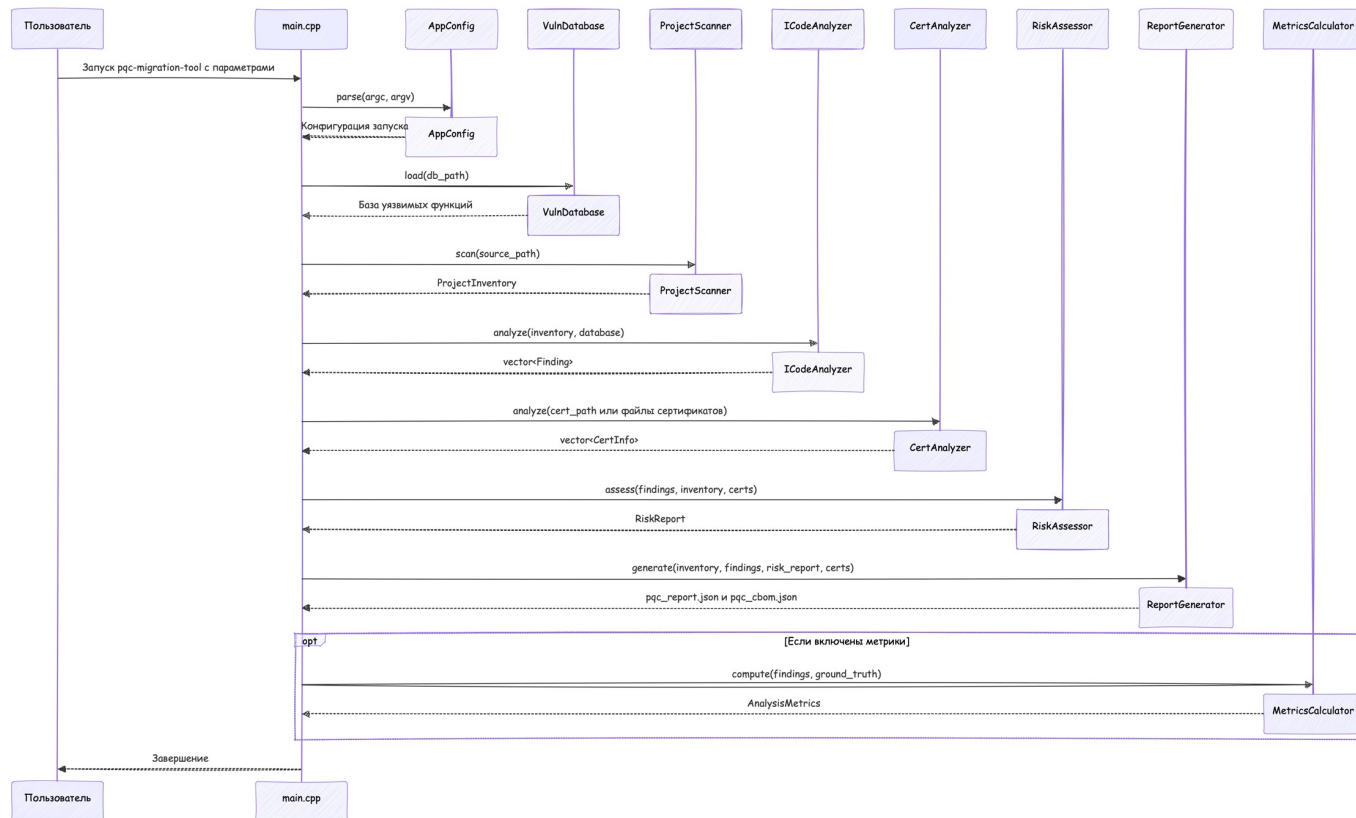


Рисунок 2 – Временная диаграмма процесса выполнения

Пример работы: ГОСТ-функция в коде

Входной фрагмент:

```
int nid = OBJ_txt2nid("id-GostR3410-2012-256");  
EvpPkeyCtxPtr ctx(EVP_PKEY_CTX_new_id(nid, GostEngine::get()));
```

Что находит инструмент:

- имя функции и библиотеку;
- используемый алгоритм;
- аргументы вызова (в режиме AST);
- базовое значение риска;
- расположение в исходном коде;
- рекомендации NIST и ТК 26.

Благодаря условному обнаружению инструмент корректно отличает применение функции с уязвимым аргументом от постквантового аналога.

Фрагмент отчета:

```
{  
  "algorithm": "RSA/EC/DH (EVP)",  
  "analyzer_mode": "ast",  
  "arguments": [  
    "OBJ_txt2nid(\"id-GostR3410-2012-256\")",  
    "nullptr"  
  ],  
  "base_risk_score": 9.0,  
  "category": "asymmetric_key_generation",  
  "col": 23,  
  "file_path": "...",  
  "function": "EVP_PKEY_CTX_new_id",  
  "library": "openssl",  
  "line": 409,  
  "nested_vulnerable_calls": [  
    "OBJ_txt2nid"  
  ],  
  "nist_reference": "...",  
  "quantum_vulnerability": "conditional",  
  "raw_line": "...",  
  "tc26_reference": "..."  
}
```


Пример работы: отчет о плане миграции

Информация по проекту:

```
{
  "file_risk_summaries": {
    "...": {
      "avg_risk_score": 7.200000000000002,
      "file_path": "...",
      "finding_count": 30,
      "functions": [
        "EVP_PKEY_CTX_new_id",
        "OBJ_txt2nid",
        "EVP_get_cipherbyname"
      ],
      "max_risk_score": 10.0,
      "overall_priority": "critical"
    }
  }
}

"nist_readiness": "not-started",
"overall_risk_score": 10.0,
"summary": {
  "critical": 15,
  "high": 0,
  "low": 7,
  "medium": 8
}
```

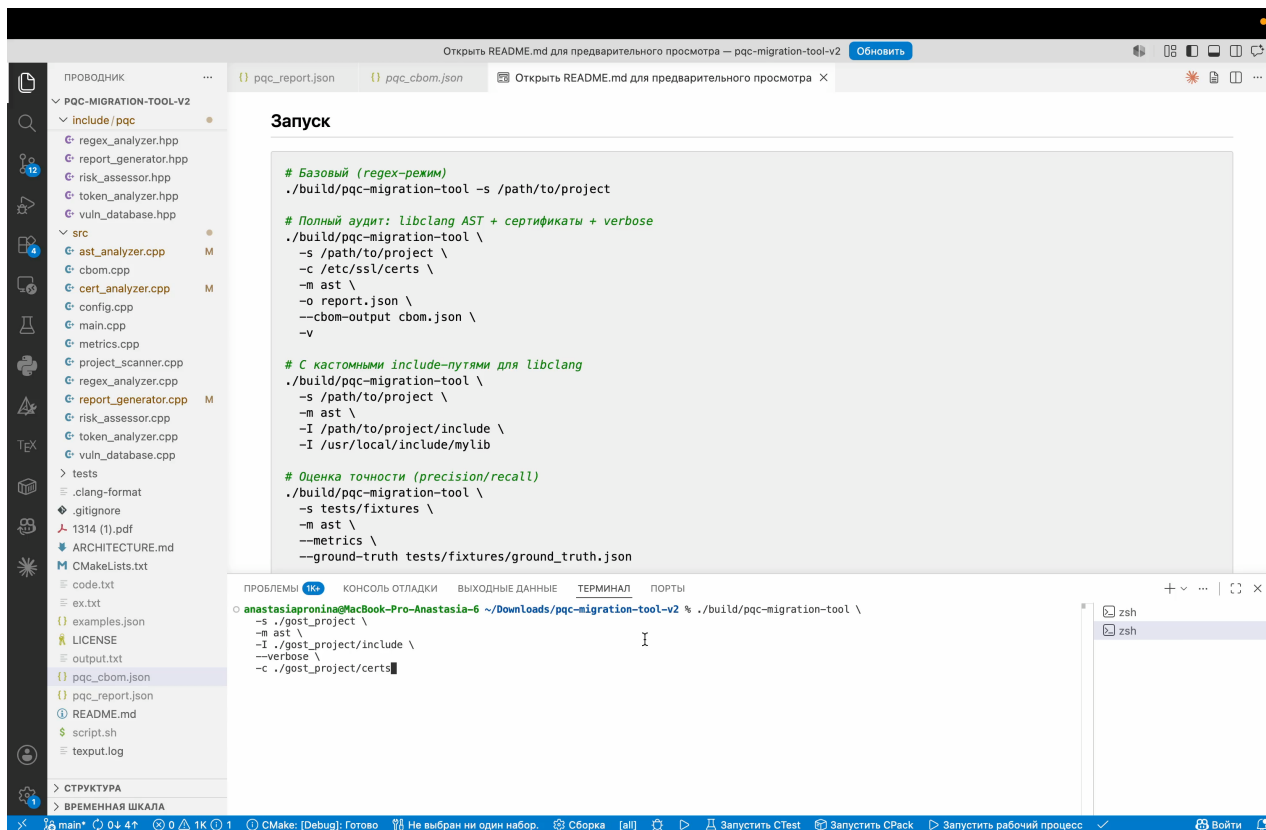
План замены функции:

```
{
  "context": {
    "function": "key_material",
    "key_material": true,
    "network_facing": false,
    "persistent_data": false,
    "test_code": false
  },
  "current_function": "EVP_PKEY_CTX_new_id",
  "current_library": "openssl",
  "file_path": "...",
  "line_number": 272,
  "migration_notes": "...",
  "migration_order": 2,
  "nist_reference": "...",
  "priority": "critical",
  "replacement": {
    "function": "...",
    "library": "..."
  }
}
```

Информация о сертификате:

```
{
  "file_path": ".../rsa2048_cert.pem",
  "is_expired": false,
  "is_self_signed": true,
  "issuer": "CN = CryptoDemo",
  "key_size_bits": 2048,
  "not_after": "Apr 19 10:13:04 2027 GMT",
  "not_before": "Apr 19 10:13:04 2026 GMT",
  "public_key_algorithm": "RSA",
  "quantum_risk": "high",
  "risk_explanation": "...",
  "risk_score": 9.0,
  "serial_number": "01",
  "sig_algorithm": "sha256WithRSAEncryption",
  "subject": "CN = CryptoDemo",
  "tc26_note": "..."
}
```

Демонстрация работы



Практический результат: экономия времени

Задача: Необходимо провести криптографическую инвентаризацию проекта. Он состоит из 23 файлов C/C++ (≈ 7000 строк исходного кода).

Результат: в процессе было обнаружено **573** вызова различных криптографических функций (OpenSSL + ГОСТ). Из них **44** оказались квантово-уязвимыми.

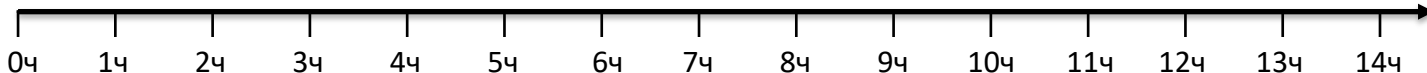
Ручной поиск:

$\approx 18-20$ часов

Утилита:



≈ 1 с. (regex) / ≈ 13 с. (AST)



Что входит в ручной поиск: grep по именам функций OpenSSL/Crypto++/ГОСТ, ручная сверка с документацией, классификация по типу, сопоставление со списками NIST, оценка контекста.

Ускорение: $\geq 5 \cdot 10^3$ раз при работе утилиты в AST-режиме на проекте сравнимого масштаба.

Метрики качества: AST vs Regex

Эталонный набор: 32 размеченные находки в тестовом файле (8 тематических секций + блок SAFE).

Метрика	AST	Regex
Эталон	32	32
TP	32	32
FP	2	8
FN	0	0
Precision	94.1%	80.0%
Recall	100%	100%
F1	97.0%	88.9%
Время	≈ 1017 мс.	≈ 113 мс.

Почему AST-режим лучше Regex

В AST-режиме анализируется семантика кода, а не просто текст: переменные трассируются и в зависимости от контекста анализатор понимает, уязвима функция или нет. За счет проверки аргументов AST-режим дает гораздо меньше ложноположительных результатов.

Причины оставшихся FP в AST

Такая ситуация неизбежно возникает в том случае, если анализатор не смог выполнить полную трассировку. Поскольку функция есть в базе, и мы не можем точно определить ее аргументы, она считается уязвимой по умолчанию.

- Разработан инструмент автоматического планирования миграции на алгоритмы постквантовой криптографии с двумя режимами анализа исходного кода (regex и AST), анализом сертификатов и генерацией отчетов JSON/CBOM CycloneDX 1.5.
- AST-режим: точность 94%, полнота близка к 100% на тестируемой базе.
- Поддерживаются стандарты NIST PCQ, ГОСТ и нормативная база ТК 26.

Все пять задач выполнены, **цель работы достигнута.**

Спасибо за внимание!

ITMO *re than a*
UNIVERSITY

Ссылка на публичный репозиторий:
<https://github.com/ProshkaAnastasia/PQCMigrationTool>